backend/src/services/observability.service.js

```
/**
 * @file Servicio de observabilidad y monitoreo de la aplicación
 * @description Proporciona funcionalidades de logging, métricas Pro
 * @requires prom-client - Client de Prometheus para métricas
 * @requires winston - Logger estructurado
 * @requires winston-daily-rotate-file - Plugin para rotación de log
 */

const client = require('prom-client');
const winston = require('winston');
const DailyRotateFile = require('winston-daily-rotate-file');
const path = require('path');

/**
 * Configuración de Prometheus Metrics
 */
const register = new client.Registry();

// Métricas HTTP
const httpRequestDuration = new client.Histogram({
  name: 'http_request_duration_seconds',
  help: 'Duration of HTTP requests in seconds',
  labelNames: ['method', 'route', 'status_code'],
  registers: [register]
});

const httpRequestCount = new client.Counter({
  name: 'http_requests_total',
  help: 'Total number of HTTP requests',
  labelNames: ['method', 'route', 'status_code'],
  registers: [register]
});

const httpErrorCount = new client.Counter({
  name: 'http_errors_total',
  help: 'Total number of HTTP errors',
  labelNames: ['method', 'route', 'error_type'],
  registers: [register]
});

// Métricas de autenticación
const authAttempts = new client.Counter({
  name: 'auth_attempts_total',
  help: 'Total authentication attempts',
  labelNames: ['endpoint', 'result'],
  registers: [register]
});

// Métricas de base de datos
const mongoQueryDuration = new client.Histogram({
  name: 'mongo_query_duration_seconds',
  help: 'Duration of MongoDB queries in seconds',
```

```
  labelNames: ['operation', 'collection'],
  registers: [register],
  buckets: [0.001, 0.01, 0.1, 0.5, 1, 2, 5]
});

const mongoQueryCount = new client.Counter({
  name: 'mongo_queries_total',
  help: 'Total number of MongoDB queries',
  labelNames: ['operation', 'collection', 'status'],
  registers: [register]
});

/**
 * Configuración de Winston Logger
 */
const logsDir = path.join(__dirname, '..', 'logs');

const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp({ format: 'YYYY-MM-DD HH:mm:ss' }),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),
  defaultMeta: { service: 'mindcare-api' },
  transports: [
    // Console output en desarrollo
    new winston.transports.Console({
      format: winston.format.combine(
        winston.format.colorize(),
        winston.format.printf(({ timestamp, level, message, ...meta
          const metaStr = Object.keys(meta).length ? JSON.stringify(
          return `${timestamp} [${level}]: ${message} ${metaStr}`;
        })
      )
    })
  ]
});

// Agregar rotación de logs en producción
if (process.env.NODE_ENV === 'production') {
  logger.add(new DailyRotateFile({
    filename: path.join(logsDir, 'application-%DATE%.log'),
    datePattern: 'YYYY-MM-DD',
    maxSize: '20m',
    maxDays: '14d',
    format: winston.format.json()
  }));

  logger.add(new DailyRotateFile({
    filename: path.join(logsDir, 'error-%DATE%.log'),
    level: 'error',
    datePattern: 'YYYY-MM-DD',
    maxSize: '20m',
    maxDays: '30d',
    format: winston.format.json()
  }));
}

/**
 * Middleware para registrar métricas HTTP
 * @param {Object} req - Objeto de request de Express
 * @param {Object} res - Objeto de response de Express
 * @param {Function} next - Función next de Express
```

```
 */
const metricsMiddleware = (req, res, next) => {
  const start = Date.now();

  // Patrones de rutas que se deben ignorar
  const ignorePaths = ['/api/metrics', '/api/health', '/health'];
  if (ignorePaths.includes(req.path)) {
    return next();
  }

  // Interceptar el método send para capturar status code
  const originalSend = res.send;
  res.send = function(data) {
    const duration = (Date.now() - start) / 1000;
    const statusCode = res.statusCode;
    const route = req.route?.path || req.path;
    const method = req.method;

    httpRequestDuration.observe(
      { method, route, status_code: statusCode },
      duration
    );
    httpRequestCount.inc({
      method,
      route,
      status_code: statusCode
    });

    if (statusCode >= 400) {
      httpErrorCount.inc({
        method,
        route,
        error_type: statusCode >= 500 ? 'server_error' : 'client_err
      });
    }

    logger.info('HTTP Request', {
      method,
      path: req.path,
      statusCode,
      duration: `${duration.toFixed(3)}s`,
      ip: req.ip
    });

    return originalSend.call(this, data);
  };

  next();
};

/**
 * Instrumentación para MongoDB con Mongoose
 * @param {Object} schema - Schema de Mongoose
 */
const instrumentMongoDB = (schema) => {
  schema.pre(/.*/, function(next) {
    this._startTime = Date.now();
    next();
  });

  schema.post(/.*/, function() {
    if (this._startTime) {
      const duration = (Date.now() - this._startTime) / 1000;
      const operation = this.op;
```

```
      const collection = this.model?.collection?.name || 'unknown';

      mongoQueryDuration.observe(
        { operation, collection },
        duration
      );

      mongoQueryCount.inc({
        operation,
        collection,
        status: 'success'
      });

      logger.debug('MongoDB Query', {
        operation,
        collection,
        duration: `${duration.toFixed(3)}s`
      });
    }
  });

  schema.post(/.*/, function(error, doc, next) {
    if (error) {
      const collection = this.model?.collection?.name || 'unknown';
      const operation = this.op;

      mongoQueryCount.inc({
        operation,
        collection,
        status: 'error'
      });

      logger.error('MongoDB Query Error', {
        operation,
        collection,
        error: error.message
      });

      next(error);
    } else {
      next();
    }
  });
};

/**
 * Registrar intento de autenticación
 * @param {string} endpoint - Endpoint de autenticación
 * @param {string} result - Resultado ('success' o 'failure')
 */
const logAuthAttempt = (endpoint, result) => {
  authAttempts.inc({ endpoint, result });
  logger.info('Authentication Attempt', { endpoint, result });
};

/**
 * Obtener métricas en formato Prometheus
 * @returns {Promise<string>} Métricas formateadas
 */
const getMetrics = async () => {
  return await register.metrics();
};

module.exports = {
```

```
245.   logger,
246.   metricsMiddleware,
247.   instrumentMongoDB,
248.   logAuthAttempt,
249.   getMetrics,
250.   register,
251.   // Exportar métricas individuales para acceso directo
252.   httpRequestDuration,
253.   httpRequestCount,
254.   httpErrorCount,
255.   authAttempts,
256.   mongoQueryDuration,
257.   mongoQueryCount
258. };
259.
```